

How tomorrow's cases get allocated

Every night at 8 PM IST the planner decides which borrowers each field agent will visit tomorrow, and in what order. This is what it actually does — read off the code, not the roadmap.

`planner_service.py``global_allocator.py``ml/visit_priority.py``ml/empirical_bayes.py``core/routing.py``verified 2026-09-03`

What runs, and when

One scheduled task per night, plus a manual trigger for managers.

```
celery beat "nightly-ml-allocation" 20:00 IST daily
└─ tasks.allocation.run_nightly_allocation(strategy="SMART")
   └─ PlannerService.plan_next_day(plan_date=)

manual POST /api/v1/manager/allocation/plan
```

It runs after the evening data ingest (~7:30 PM) and after repayment scoring, so it reads the day's fresh DPD and payment state. Planning is scoped to one manager's team: their agents, and cases either already held by those agents or unassigned.

The manager picks one of three objectives — `BALANCED` , `MAX_RECOVERY` , `MIN_DISTANCE` — which changes the scoring weights and nothing else.

The pipeline

Three stages, in this order. The numbering is the actual sequence — each stage consumes the previous one's output.

Build the pool, then rank it

Candidate cases are open (not `PAID`, `CLOSED` or `WRITTEN_OFF`) with `collected_amount < target_amount`. Two filters then remove cases that must not be visited tomorrow, each recorded as a decision so the manager sees why a case is absent rather than it silently disappearing:

- `DEFERRED_PTP` — an active promise to pay falls due after tomorrow. Arriving early collects nothing and harasses someone who is cooperating.
- `DEFERRED_VISIT_CAP` — the case has already used `max_visits_allowed` visits *this calendar month*. A contact-frequency control, reset on the 1st, not a lifetime budget.

Survivors are scored 0–100 for visit priority and sorted highest first. That score is a hand-weighted scorecard of three components — value, urgency, effort — described below.

Assign cases to agents — globally, in one solve

Each agent is expanded into `max_cases_per_day` discrete slots, giving a cases × slots matrix. Every eligible pair gets a utility score; ineligible pairs get a `1e6` penalty. The matrix is then solved by the **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`) for the assignment that maximises total utility across the whole team at once.

Cases that end up assigned but sit more than 22 km from their agent's base are pulled back out as `DEFERRED_ROUTE_INFEASIBLE`. Anything left with no slot becomes `DEFERRED`.

Sequence each agent's day into a beat

Per agent, the assigned stops are ordered by `optimize_route()`: a travel-time matrix from **OSRM** (real road times) fed into an **OR-Tools** routing solver using `PATH_CHEAPEST_ARC`. If OSRM is unreachable it falls back to a Haversine time estimate; if OR-Tools fails, to nearest-neighbour.

The result is a `Beat` row with an ordered case list, estimated distance (crow-flies × 1.15 for urban road network) and duration (25 km/h + 20 min per stop).

The scoring function

One weighted sum decides every case–agent pairing. Six terms, each scaled 0–1, weights set by the manager's objective.

```

utility =  $\sum$  weight[term] × score[term]

weighted_utility = utility × (1 + priority/100 × 0.25) ← PRIORITY_UPLIFT
cost_matrix[case, slot] = -weighted_utility

```

OBJECTIVE WEIGHT TABLE — GET_OBJECTIVE_WEIGHTS()

TERM	BALANCED	MAX RECOVERY	MIN DISTANCE
expected_recovery	0.45	0.70	0.05
proximity	0.40	0.10	0.80
skills	0.05	0.10	0.05
workload	0.05	0.05	0.05
continuity	0.05	0.05	0.05
language	0.05	0.05	0.05

How each term is computed

```

expected_recovery
    collectable = target_amount - collected_amount
    prob        = clamp(affinity × (0.80 + 0.20 × tier_weight), 0.20, 0.85)
    score       = log1p(collectable × prob / 25,000) / log1p(300,000 / 25,000)

proximity
    effective_km = min(km from agent base, km from centroid of agent's open cases)
    score        = exp(-effective_km / 5.5)

skills
    score = 0.6 × tier_weight + 0.4 × spec_match
           tier_weight  TIER_1 1.0 · TIER_2 0.8 · TIER_3 0.6
           spec_match   1.0 if agent specialisation = loan type, else 0.5

workload
    score = max(0.1, 1 - slot_index / max_cases_per_day)

continuity
    score = 1.0 if the case is already this agent's, else 0.0

language
    score = 1.0 if borrower's language ∈ agent.languages_spoken, else 0.0

```

affinity is the Empirical Bayes estimate — the only fitted quantity in the whole function. The value term is *logarithmic* because the book has a heavy right tail; a linear scale let one large case outrank everything else and effectively overrode the manager's objective switch.

Each term's weight × score is its share of the decision. Those products are stored on the `AllocationDecision` row, which is what lets the audit panel rank the reasons a case went to a particular agent instead of dumping raw factors and leaving the manager to guess.

Hard gates

Applied before scoring. A gate either removes the case entirely, or removes one agent from that case's row — which lets the solve pick the next best agent on its own, with no reassignment pass.

GATES, IN EVALUATION ORDER

GATE	EFFECT	OUTCOME RECORDED
Borrower on Do-Not-Contact	Removes the case	BLOCKED
Borrower flagged hostile	Removes the case	BLOCKED
Borrower requires a female agent	Removes non-female agents from the row	—
PTP fatigue — 3 broken promises to the same agent	Removes that agent from the row	—
Borrower beyond the 16 km territory radius from agent base	Removes that agent from the row	—
Assigned stop > 22 km from base (checked after the solve)	Pulls the case back out	DEFERRED_ROUTE_INFEASIBLE

What is a model, and what isn't

Three techniques are in the loop. Only one of them is fitted from data.

Hungarian bipartite assignment — combinatorial optimisation

Not machine learning. It is an exact algorithm that finds the globally optimal assignment in polynomial time. It is used because the alternative — walking the priority-sorted list and giving each case its best available agent — is greedy: an early case takes an agent a later, better-matched case needed, and the total is worse. Solving all pairs at once removes the ordering effect entirely.

Empirical Bayes shrinkage — a fitted statistical model

This is the one model in the allocation path. It estimates how well each agent recovers within a *segment*, where a segment is (loan type × DPD bucket), and

shrinks that estimate toward the segment average in proportion to how little evidence supports it:

$$\hat{w} = (n / (n + k)) \times \bar{w} + (k / (n + k)) \times \mu_{\text{segment}} \quad k = 10$$

n distinct (agent, case) observations in the last 180 days
 $\bar{w}$ that agent's own recovered ÷ target in the segment
 μ the segment's own average across all agents

It is used because agents have very few observations per segment, and a raw recovery rate over three cases is noise. Shrinkage means an agent with real history is differentiated, and an agent without one degrades to the segment average instead of being scored on an accident. Below 5 observations the estimate *is* the segment average, by design.

Visit priority — a scorecard, not a model

Three hand-weighted components summed to 0–100: **value** (a doubling ladder on recoverable amount), **urgency** (peaks in the fortnight before the 90-day NPA line, then declines — the classification damage is already done), and **effort** (a penalty that grows with visits already spent, since a first visit historically returned ~₹6,667 against ~₹2,872 by the third).

The weights were chosen by hand and were never fitted. The file says so itself, returning `"is_modelled": False` on every score.

Limitations

Measured against the live book on 2026-09-03, not estimated.

No agent has a recorded gender, so 30 cases can never be allocated

The female-agent gate tests `agent.gender`. The column exists and is nullable — and every agent row is null, so the test is never true. Every agent is therefore removed from the row of any borrower who requires a female agent, and the case falls through to the end of the solve.

agents with gender set	0 of 18
borrowers requiring female	31
their open cases	30

Those cases are recorded as **DEFERRED** with the reason *"all nearby agents reached daily capacity limit"* — which is not what happened. This is a data-completeness problem being reported as a capacity problem.

The affinity term barely differentiates agents

Empirical Bayes needs 5 observations in a segment before it returns anything other than the segment average. On the current book almost nothing clears that bar:

(agent, segment) cells	315	
with $n \geq 5$	7	(2.2%)
with $n \geq 20$	0	
segment prior (global)	0.8194	

So for roughly 98% of pairings, `affinity` is the segment average — identical for every agent in that segment. The term carrying 45–70% of the utility is, in practice, mostly ranking cases rather than distinguishing agents. That is the estimator behaving correctly with thin data, not a bug, but it should be understood before anyone reads agent-fit into the output.

Distance in the assignment is crow-flies, not road

Both the territory gate and the proximity score use Haversine. Real road distance enters only at stage 3, when the route is sequenced — by which point the set of cases has already been chosen. A river or a railway line between agent and borrower is invisible to the decision that matters.

The 22 km check runs after the solve, so it wastes a slot

A case beyond the route-feasibility boundary is scored, assigned a slot, and only then pulled back out. The slot is not returned to the pool, so another case that could have been visited is not.

Every beat is stamped as ML-generated

Beat rows are written with `is_ml_generated=True` and `ml_model_version="planner-smart-v1.0"` regardless of what produced them. There is no trained model in this path — the naming implies one, and the legacy strategy gets the same stamp.

Smaller things worth knowing

- **Off-duty agents are planned for.** The pool excludes only `SUSPENDED` and `ON_LEAVE` ; `OFF_DUTY` agents still receive beats.
- **Hostile borrowers are dropped, not routed differently.** `BLOCKED` means nobody visits them, indefinitely — there is no escalation path out of that state in the planner.
- **Recovery probability has a floor of 0.20.** No pairing is ever forecast to recover less than a fifth of the outstanding balance, so expected-recovery totals are optimistic by construction.
- **Language matching is exact string membership.** No dialect or script normalisation.
- **OSRM defaults to the public demo server.** Fine for a demo; rate-limited and unsuitable for production volume.
- **Nothing learns from the outcome.** Whether yesterday's plan actually collected is never fed back into the weights. The only adaptive element is Empirical Bayes, which updates as payments accumulate.

How to improve it

Ordered by cost against benefit, cheapest first.

Populate agent gender

A data fix, not a code change. It unblocks 30 cases immediately. Separately, the fallthrough reason should distinguish "no eligible agent" from "capacity full" — reporting the wrong cause is how this stayed invisible.

Move the 22 km check before the solve

Fold the route-feasibility boundary into the eligibility gates alongside the 16 km territory radius. Infeasible pairs then never consume a slot, and the audit trail gains nothing it did not already record.

Give the value term road distance

A cached OSRM matrix between agent bases and borrower coordinates, refreshed periodically rather than inside the solve, would make proximity mean travel time instead of straight-line kilometres — with no change to the solver.

Replace the fixed probability with a calibrated one

`prob_recovery` is currently $\text{agent skill} \times \text{tier}$, clamped. A trained borrower-side model would replace it with a per-case probability that knows the borrower's DPD, arrears ratio, payment recency and promise history — none of which the

current term sees. The pipeline for this is built and validated end to end on synthetic data; it is blocked on real labels, which arrive 30 days after the first scored snapshots.

A caveat on that last one, from the validation run: on synthetic data the trained model did *not* beat the existing scorecard at ranking — ROC 0.6355 against 0.6312, identical lift. Its clear advantage was calibration. So the honest first use of a borrower model here is improving the *expected-recovery forecast*, not reordering the visit queue.

Close the feedback loop

Every allocation decision is already stored with its full score breakdown, and every visit outcome is recorded against the case. Joining the two would show which weights actually correlate with collection — turning the objective table from a set of chosen constants into something measured. Nothing in the schema prevents this today; it simply has not been done.

Every figure and formula on this page was read from the source or measured against the live database on 2026-09-03. Where a number is a threshold or a weight, it is the constant in the code, not a target. Nothing here describes planned work except where the section says so explicitly.